

Theory of Electromagnetic Fields, LAB 2

Name: He Yanning

Student ID: 12311512

Southern University of Science and Technology

Abstract—This experiment studies the electrostatic potential distribution produced by a uniform line charge from A(-1,0) to B(1,0), with linear charge density $\rho = 1 \times 10^{-9} \text{C/m}$. The analytical integral solution is first used as the continuous reference solution. Then the line charge is divided into 20, 50, and 100 discrete segments, and the method of infinitesimals is applied. The potential surface, equipotential contours, electric field lines, and quantitative errors are compared. The results show that increasing the number of segments greatly improves the numerical accuracy. The RMS error decreases from 0.7666 for N=20 to 0.00497 for N=100.

I. Introduction

The objectives of this experiment are to achieve proficiency in using Matlab for calculating the electric field distribution of linear charges and generating the corresponding graphical representations, while developing a deeper conceptual understanding of the method of infinitesimals and the principles of calculus through practical application.

II. METHOD

The electric field intensity E produced by a point charge in a vacuum is given by:

$$E = k \frac{Q}{R^2} \hat{r} \quad (1)$$

where the coefficient $k = 9 \times 10^9 \text{N} \cdot \text{m}^2/\text{C}^2$ is the electrostatic constant, Q represents the quantity of the point charge, and R is the distance from the point charge to the field point.

If the potential at infinity is taken as the zero reference point, the electric potential Φ generated by a point charge in a vacuum is:

$$\Phi = k \frac{Q}{R} \quad (2)$$

The electric field intensity can also be expressed as the negative gradient of the electric potential:

$$E = -\nabla\Phi \quad (3)$$

The total electric potential generated by N point charges in a vacuum is the algebraic sum:

$$\Phi = \sum_{i=1}^N k \frac{Q_i}{R_i} \quad (4)$$

Similarly, the total electric field intensity generated by N point charges can be derived using Equation (3).

$$\begin{aligned} V &= k \int_{-1}^1 \frac{\rho dx}{R} \\ &= k \int_{-1}^1 \frac{\rho dx}{\sqrt{(x - X_0)^2 + Y_0^2}} \\ &= k\rho \ln \left| (x - X_0) + \sqrt{(x - X_0)^2 + Y_0^2} \right| \Big|_{-1}^1 \\ &= k\rho \ln \left(\frac{1 - X_0 + \sqrt{(1 - X_0)^2 + Y_0^2}}{-1 - X_0 + \sqrt{(-1 - X_0)^2 + Y_0^2}} \right) \end{aligned}$$

When the field source is a continuously distributed charge, such as a linear charge, we can employ the method of infinitesimals (element-based method) or calculus to solve the problem. The specific steps for the method of infinitesimals are as follows:

1 Segmentation: Divide the linear charge into N small segments (typically using a uniform subdivision).

2 Point Charge Approximation: Treat each small segment as a point charge and use Equation (2) to calculate the potential it generates in space.

3 Superposition: Apply Equation (4) to sum the potentials of all segments, obtaining the total potential of the entire linear charge.

4 Field Calculation: Use Equation (3) to determine the total electric field intensity.

Results obtained via the method of infinitesimals inherently contain numerical errors. This error primarily depends on the number of segments defined in Step 1; generally, a higher number of segments leads to a smaller error. For certain geometries, the exact electric field distribution can be solved analytically using calculus. This allows for a comparative study of the relationship between the discretization error and the number of segments used.

III. EXPERIMENT

Objective: Use Matlab to analyze the electrostatic field distribution of a linear charge in a 2D Cartesian coordinate system.

Parameters: A linear charge with a density of $\rho = 1 \times 10^{-9} \text{C/m}$ is distributed along the line segment between point A(-1, 0) and point B(1, 0). (All coordinates are in meters).

A. Analytical Solution (The "True" Distribution)

Using the method of calculus, calculate the theoretical electric potential distribution at any point (X_0, Y_0) in the 2D plane. The derivation process is as follows:

- Differential Element: Let an infinitesimal charge element $dQ = \rho dx$ be located at $(x, 0)$ on the x-axis, where $x \in [-1, 1]$.
- Distance: The distance R from the element to the field point (X_0, Y_0) is:
$$R = \sqrt{(X_0 - x)^2 + Y_0^2}$$
- Integral Setup: The potential $d\Phi$ generated by this element is:
$$d\Phi = \frac{k\rho dx}{\sqrt{(X_0 - x)^2 + Y_0^2}}$$
- Total Potential: Integrating from $x = -1$ to $x = 1$:
$$\Phi(X_0, Y_0) = \int_{-1}^1 \frac{k\rho}{\sqrt{(X_0 - x)^2 + Y_0^2}} dx$$

Task: Select an appropriate field domain. Based on the analytical result above, develop a Matlab program to calculate and plot the following within the domain:

- 3D surface plot of the electric potential distribution.
- Equipotential contour lines (select appropriate intervals).
- Electric field lines (represented by smooth, continuous curves).

B. Numerical Solution (Method of Infinitesimals)

Using the method of infinitesimals, divide the linear charge into 20, 50, and 100 segments, respectively. Using the same field domain as Step 1, use Matlab to calculate and plot:

- The potential distribution, equipotential lines, and electric field lines for each discretization level.

C. Quantitative Error Analysis

Quantitatively compare the results obtained from the method of infinitesimals against the analytical "true" values. Conduct a study on the relationship between the calculation error and the number of segments (N) used in the discretization.

IV. CODE and EXPERIMENT

A. Code 1: Continuous Line Charge Simulation

```
clc; clear; close all;

%% 1. Parameter Initialization & Domain Setup
k = 9e9; % Electrostatic constant (N·m2/C2)
p = 1e-9; % Linear charge density (C/m)
xm = 2; ym = 2; % Define observation domain boundaries (meters)

% Generate calculation grid for the 2D plane
% Increasing resolution to 200 for smoother gradient transitions
x = linspace(-xm, xm, 200);
y = linspace(-ym, ym, 200);
```

```
[X, Y] = meshgrid(x, y);
```

```
%% 2. Analytical Solution (Integral Method)
% Based on the integral formula:  $V = k \cdot p \cdot \ln \left[ \frac{r_1}{r_2} \right]$ 
% where the integration is performed along the segment  $x' = [-1, 1]$ 
r1 = 1 - X + sqrt((1 - X).^2 + Y.^2);
r2 = -1 - X + sqrt((-1 - X).^2 + Y.^2);
V = k .* p .* log(r1 ./ r2);
```

```
%% 3. Plotting 3D Potential Distribution
figure('Color', 'w'); % Set background to white for report compatibility
mesh(X, Y, V);
colormap jet; % Apply heat-map color scheme
grid on;
xlabel('X-axis (m)'); ylabel('Y-axis (m)');
zlabel('Potential V (V)');
% Title formatted as a cell array for multi-line display
title({'Potential Distribution of Line Charge (Analytical)', ...
      'Author: Yanning He, ID: 12311512'});
saveas(gcf, 'figure1.1.jpg');
```

```
%% 4. Plotting Equipotential Contours
figure('Color', 'w');
V_levels = linspace(0, 100, 50); % Define contour density
[C, h] = contour(X, Y, V, V_levels, 'LineWidth', 1);
```

```
hold on;

% Highlight the linear charge source (Red solid line)
plot([-1, 1], [0, 0], 'r', 'LineWidth', 3);
grid on;
axis equal; % Maintain 1:1 aspect ratio to avoid spatial distortion
xlabel('X-axis (m)'); ylabel('Y-axis (m)');
title({'Equipotential Contours (Analytical Method)', ...
      'Author: Yanning He, ID: 12311512'});
saveas(gcf, 'figure1.2.jpg');
```

```
%% 5. Plotting Composite Map (Electric Field Lines & Potential)
% Calculate electric field intensity  $E = -\text{grad}(V)$ 
[Ex, Ey] = gradient(-V);
```

```
figure('Color', 'w');
hold on;

% Define seed points for streamlines to ensure uniform distribution
xs_seed = -1.2:0.2:1.2;
ys_seed_pos = 0.05 * ones(size(xs_seed));
ys_seed_neg = -0.05 * ones(size(xs_seed));

% Generate streamlines to represent the electric field
```

```

h1 = streamline(X, Y, Ex, Ey, xs_seed,
    ys_seed_pos);
h2 = streamline(X, Y, Ex, Ey, xs_seed,
    ys_seed_neg);
set([h1, h2], 'LineWidth', 2, 'Color', [0.1,
    0.4, 0.1]);

% Overlay contours and the physical charge
location
contour(X, Y, V, V_levels, 'LineColor', 'b');
plot([-1, 1], [0, 0], 'r', 'LineWidth', 4);

grid on;
axis equal; % Critical for
    preserving orthogonality of E and V lines
xlabel('X-axis (m)'); ylabel('Y-axis (m)');
title({'Electric Field Lines & Equipotential
    Mapping', ...
    'Author: Yanning He, ID: 12311512'});
saveas(gcf, 'figure1.3.jpg');

```

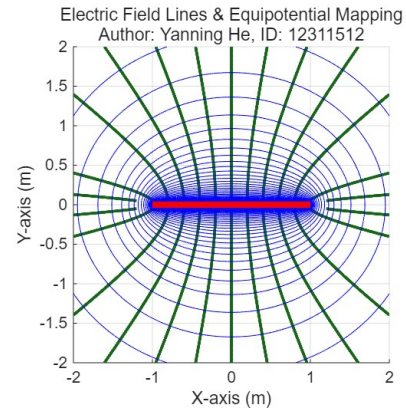


Fig. 3. Electric Field Lines & Equipotential Mapping

B. Code 2: Segmented charge simulation of 20, 50, 100
N=20

```

clc; clear; close all;

%% 1. Parameter Initialization &
    Discretization Setup
k = 9e9; % Electrostatic
    constant (N·m^2/C^2)
p = 1e-9; % Linear charge
    density (C/m)
L_total = 2; % Total length of the
    charge (from -1 to 1)
N = 20; % Number of segments (
    Change to 50 or 100 for other tasks)
dL = L_total / N; % Length of each
    infinitesimal element
dQ = p * dL; % Charge of each
    infinitesimal element

% Observation domain setup
xm = 2; ym = 2;
x = linspace(-xm, xm, 100);
y = linspace(-ym, ym, 100);
[X, Y] = meshgrid(x, y);

%% 2. Method of Infinitesimals (Numerical
    Summation)
V = zeros(size(X)); % Initialize potential
    matrix
for i = 1:N
    % Calculate the center position of each
    segment
    % Start at -1, offset by half a segment,
    then increment by dL
    x_pos = -1 + (i - 0.5) * dL;

    % Distance from the segment center to all
    grid points
    R = sqrt((X - x_pos).^2 + Y.^2);

    % Superposition of potential: V = sum( k *
    dQ / R )
    V = V + (k * dQ ./ R);
end

%% 3. Plotting 3D Potential Distribution

```

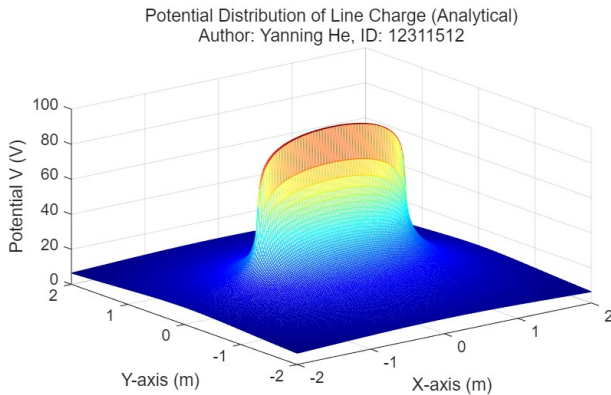


Fig. 1. Potential Distribution of Line Charge (Analytical)

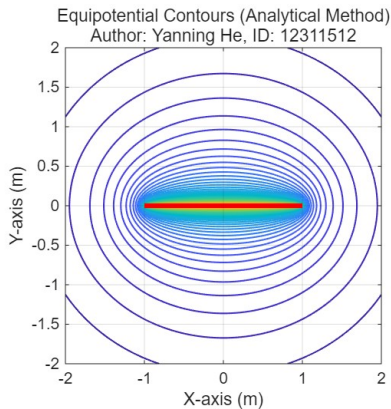


Fig. 2. Equipotential Contours (Analytical Method)

```

figure('Color', 'w');
mesh(X, Y, V);
colormap jet;
grid on;
xlabel('X-axis (m)'); ylabel('Y-axis (m)');
zlabel('Potential V (V)');
title(['Potential Distribution (Method of Infinitesimals, N=', num2str(N), ')'], ...
      'Author: Yanning He, ID: 12311512'}, 'FontSize', 12);
saveas(gcf, 'figure2.1.jpg');

%% 4. Plotting Equipotential Contours
figure('Color', 'w');
V_levels = linspace(0, 100, 101); % Contour levels from 0V to 100V
contour(X, Y, V, V_levels);
hold on;
plot([-1, 1], [0, 0], 'r', 'LineWidth', 3); % Represent the physical line charge
grid on;
axis equal;
xlabel('X-axis (m)'); ylabel('Y-axis (m)');
title(['Equipotential Lines (Method of Infinitesimals, N=', num2str(N), ')'], ...
      'Author: Yanning He, ID: 12311512'}, 'FontSize', 12);
saveas(gcf, 'figure2.2.jpg');

%% 5. Plotting Electric Field Lines
[Ex, Ey] = gradient(-V); % E-field is the negative gradient of potential
xs_seed = -1.2:0.1:1.2; % Starting points for streamlines
ys_seed_pos = 0.05 * ones(size(xs_seed));
ys_seed_neg = -0.05 * ones(size(xs_seed));

figure('Color', 'w');
hold on;
% Draw streamlines representing the direction of the electric field
h1 = streamline(X, Y, Ex, Ey, xs_seed, ys_seed_pos);
h2 = streamline(X, Y, Ex, Ey, xs_seed, ys_seed_neg);
set([h1, h2], 'LineWidth', 1.5, 'Color', [0.1, 0.4, 0.1]);

contour(X, Y, V, V_levels); % Overlay equipotential lines
plot([-1, 1], [0, 0], 'r', 'LineWidth', 3);
grid on;
axis equal;
xlabel('X-axis (m)'); ylabel('Y-axis (m)');
title(['Electric Field Lines (Method of Infinitesimals, N=', num2str(N), ')'], ...
      'Author: Yanning He, ID: 12311512'}, 'FontSize', 12);
saveas(gcf, 'figure2.3.jpg');

```

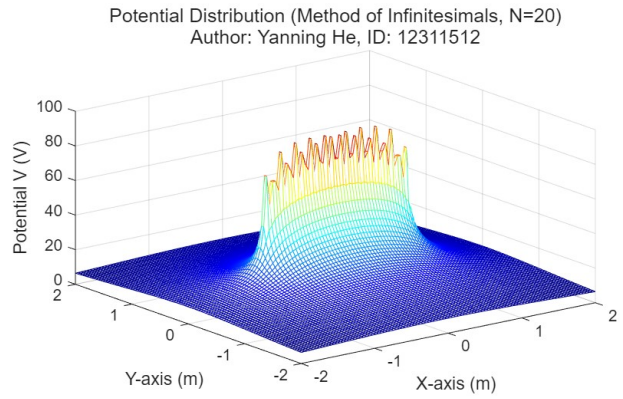


Fig. 4. Potential Distribution

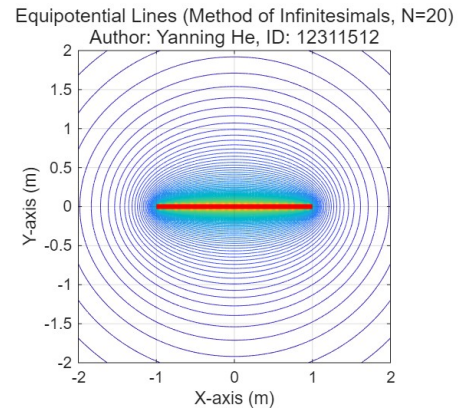


Fig. 5. Equipotential

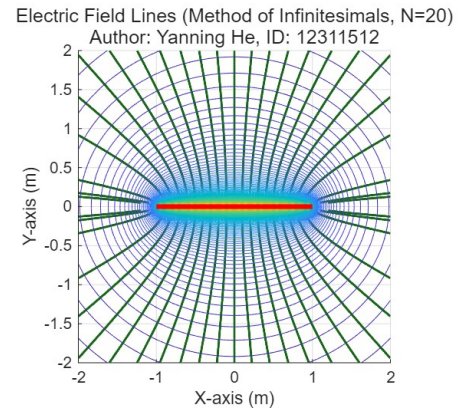


Fig. 6. Electric Field Lines

```

N=50

clc; clear; close all;

%% 1. Parameter Initialization &
    Discretization Setup
k = 9e9; % Electrostatic
    constant (N·m2/C2)
p = 1e-9; % Linear charge
    density (C/m)
L_total = 2; % Total length of the
    charge (from -1 to 1)
N = 50; % Number of segments (
    Change to 50 or 100 for other tasks)
dL = L_total / N; % Length of each
    infinitesimal element
dQ = p * dL; % Charge of each
    infinitesimal element

% Observation domain setup
xm = 2; ym = 2;
x = linspace(-xm, xm, 100);
y = linspace(-ym, ym, 100);
[X, Y] = meshgrid(x, y);

%% 2. Method of Infinitesimals (Numerical
    Summation)
V = zeros(size(X)); % Initialize potential
    matrix
for i = 1:N
    % Calculate the center position of each
    segment
    % Start at -1, offset by half a segment,
    then increment by dL
    x_pos = -1 + (i - 0.5) * dL;

    % Distance from the segment center to all
    grid points
    R = sqrt((X - x_pos).^2 + Y.^2);

    % Superposition of potential: V = sum( k *
    dQ / R )
    V = V + (k * dQ ./ R);
end

%% 3. Plotting 3D Potential Distribution
figure('Color', 'w');
mesh(X, Y, V);
colormap jet;
grid on;
xlabel('X-axis (m)'); ylabel('Y-axis (m)');
zlabel('Potential V (V)');
title({'Potential Distribution (Method of
    Infinitesimals, N=', num2str(N), ')'}, ...
    'Author: Yanning He, ID: 12311512'}, '
    FontSize', 12);
saveas(gcf, 'figure3.1.jpg');

%% 4. Plotting Equipotential Contours
figure('Color', 'w');
V_levels = linspace(0, 100, 101); % Contour
    levels from 0V to 100V
contour(X, Y, V, V_levels);
hold on;
plot([-1, 1], [0, 0], 'r', 'LineWidth', 3); %
    Represent the physical line charge
grid on;

```

```

axis equal;
xlabel('X-axis (m)'); ylabel('Y-axis (m)');
title({'Equipotential Lines (Method of
    Infinitesimals, N=', num2str(N), ')'}, ...
    'Author: Yanning He, ID: 12311512'}, '
    FontSize', 12);
saveas(gcf, 'figure3.2.jpg');

%% 5. Plotting Electric Field Lines
[Ex, Ey] = gradient(-V); % E-field is the
    negative gradient of potential
xs_seed = -1.2:0.1:1.2; % Starting points for
    streamlines
ys_seed_pos = 0.05 * ones(size(xs_seed));
ys_seed_neg = -0.05 * ones(size(xs_seed));

figure('Color', 'w');
hold on;
% Draw streamlines representing the direction
    of the electric field
h1 = streamline(X, Y, Ex, Ey, xs_seed,
    ys_seed_pos);
h2 = streamline(X, Y, Ex, Ey, xs_seed,
    ys_seed_neg);
set([h1, h2], 'LineWidth', 1.5, 'Color', [0.1,
    0.4, 0.1]);

contour(X, Y, V, V_levels); % Overlay
    equipotential lines
plot([-1, 1], [0, 0], 'r', 'LineWidth', 3);
grid on;
axis equal;
xlabel('X-axis (m)'); ylabel('Y-axis (m)');
title({'Electric Field Lines (Method of
    Infinitesimals, N=', num2str(N), ')'}, ...
    'Author: Yanning He, ID: 12311512'}, '
    12);
saveas(gcf, 'figure3.3.jpg');

```

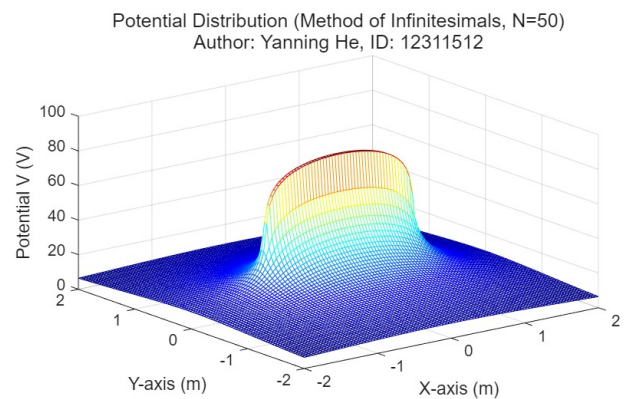


Fig. 7. Potential Distribution

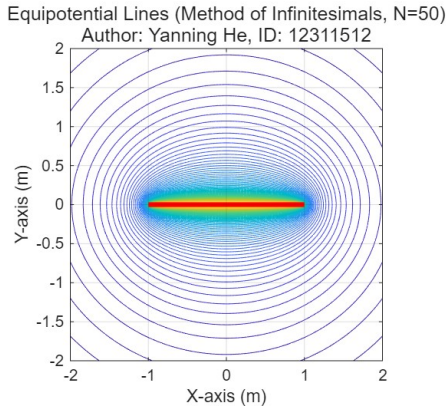


Fig. 8. Equipotential Lines

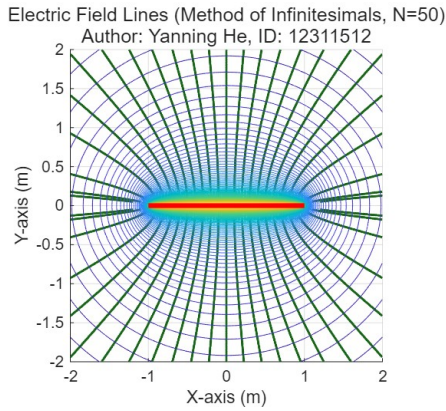


Fig. 9. Electric Field Lines

N=100

```
clc; clear; close all;

%% 1. Parameter Initialization &
    Discretization Setup
k = 9e9; % Electrostatic
        constant (N·m2/C2)
p = 1e-9; % Linear charge
        density (C/m)
L_total = 2; % Total length of the
            charge (from -1 to 1)
N = 100; % Number of segments
        (Change to 50 or 100 for other tasks)
dL = L_total / N; % Length of each
            infinitesimal element
dQ = p * dL; % Charge of each
            infinitesimal element

% Observation domain setup
xm = 2; ym = 2;
x = linspace(-xm, xm, 100);
y = linspace(-ym, ym, 100);
[X, Y] = meshgrid(x, y);

%% 2. Method of Infinitesimals (Numerical
    Summation)
V = zeros(size(X)); % Initialize potential
                    matrix
```

```
for i = 1:N
    % Calculate the center position of each
    segment
    % Start at -1, offset by half a segment,
    then increment by dL
    x_pos = -1 + (i - 0.5) * dL;

    % Distance from the segment center to all
    grid points
    R = sqrt((X - x_pos).^2 + Y.^2);

    % Superposition of potential: V = sum( k *
    dQ / R )
    V = V + (k * dQ ./ R);
end
```

```
%% 3. Plotting 3D Potential Distribution
figure('Color', 'w');
mesh(X, Y, V);
colormap jet;
grid on;
xlabel('X-axis (m)'); ylabel('Y-axis (m)');
zlabel('Potential V (V)');
title(['Potential Distribution (Method of
        Infinitesimals, N=', num2str(N), ')'], ...
        'Author: Yanning He, ID: 12311512'}, '
        FontSize', 12);
saveas(gcf, 'figure4.1.jpg');
```

```
%% 4. Plotting Equipotential Contours
figure('Color', 'w');
V_levels = linspace(0, 100, 101); % Contour
        levels from 0V to 100V
contour(X, Y, V, V_levels);
hold on;
plot([-1, 1], [0, 0], 'r', 'LineWidth', 3); %
        Represent the physical line charge
grid on;
axis equal;
xlabel('X-axis (m)'); ylabel('Y-axis (m)');
title(['Equipotential Lines (Method of
        Infinitesimals, N=', num2str(N), ')'], ...
        'Author: Yanning He, ID: 12311512'}, '
        FontSize', 12);
saveas(gcf, 'figure4.2.jpg');
```

```
%% 5. Plotting Electric Field Lines
[Ex, Ey] = gradient(-V); % E-field is the
        negative gradient of potential
xs_seed = -1.2:0.1:1.2; % Starting points for
        streamlines
ys_seed_pos = 0.05 * ones(size(xs_seed));
ys_seed_neg = -0.05 * ones(size(xs_seed));

figure('Color', 'w');
hold on;
% Draw streamlines representing the direction
of the electric field
h1 = streamline(X, Y, Ex, Ey, xs_seed,
        ys_seed_pos);
h2 = streamline(X, Y, Ex, Ey, xs_seed,
        ys_seed_neg);
set([h1, h2], 'LineWidth', 1.5, 'Color', [0.1,
        0.4, 0.1]);

contour(X, Y, V, V_levels); % Overlay
        equipotential lines
```

```

plot([-1, 1], [0, 0], 'r', 'LineWidth', 3);
grid on;
axis equal;
xlabel('X-axis (m)'); ylabel('Y-axis (m)');
title(['Electric Field Lines (Method of Infinitesimals, N=100)']);
title(['Infinitesimals, N=', num2str(N), ''], ...
'Author: Yanning He, ID: 12311512'}, ...
12);
saveas(gcf, 'figure4.3.jpg');

```

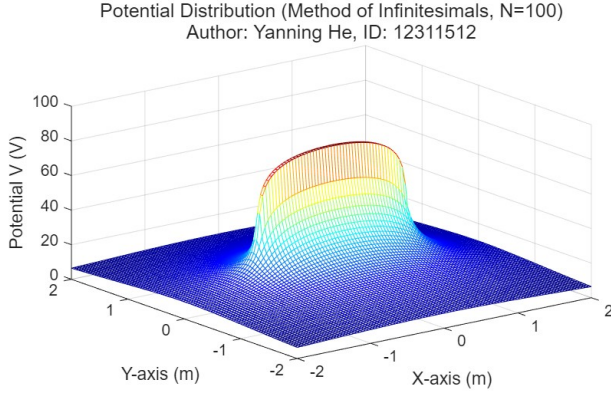


Fig. 10. Potential Distribution

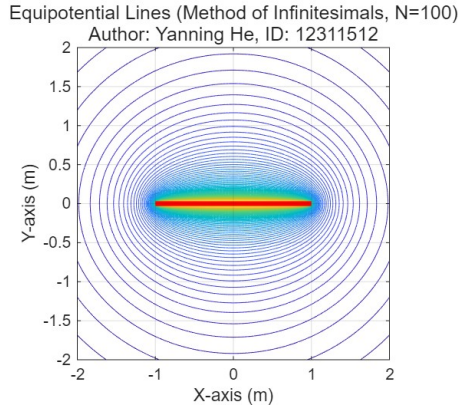


Fig. 11. Equipotential Lines

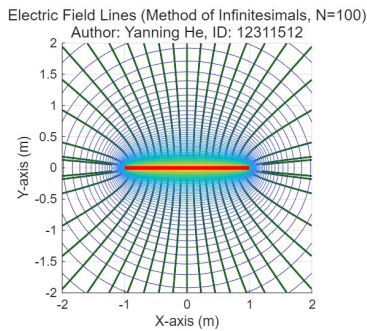


Fig. 12. Electric Field Lines

V. Analysis

TABLE I
Potential Distribution Data at Observation Points (P1–P4)

Point	x (m)	y (m)	Distance to A (m)
P1	-2	-2.0	2.2361
P2	-2	-1.5	1.8028
P3	-2	-1.0	1.4142
P4	-2	-0.5	1.1180

Point	$V_{continuous}$ (V)	$V_{N=100}$ (V)	$V_{N=50}$ (V)	$V_{N=20}$ (V)
P1	6.4220	6.4220	6.4219	6.4219
P2	7.3664	7.3664	7.3664	7.3661
P3	8.4337	8.4336	8.4335	8.4327
P4	9.4333	9.4332	9.4329	9.4310

From the preliminary qualitative analysis of the figures, the 20-segment approximation shows a relatively large deviation from the continuous analytical solution. When the number of segments increases to 50 and 100, the numerical results become much closer to the continuous solution. Four representative sampling points were selected for quantitative comparison. At the same coordinates, the potential values follow the order: continuous solution, 100 segments, 50 segments, and 20 segments. As the sampling point moves closer to the endpoint A(-1,0), the deviation between the discretized result and the continuous solution becomes larger. This indicates that the near-field region is more sensitive to charge discretization, while increasing the number of segments can effectively reduce the numerical error.

```

clc; clear; close all;

%% 1. Parameter Initialization
k = 9e9; % Electrostatic constant (N·m2/C2)
p = 1e-9; % Linear charge density (C/m)
xm = 2; ym = 2; % Observation domain boundaries (m)

% Generate calculation grid
x = linspace(-xm, xm, 100);
y = linspace(-ym, ym, 100);
[X, Y] = meshgrid(x, y);

%% 2. Reference Potential (Analytical Integral Method)
% Theoretical potential used as the "True Value" for error comparison
r1 = 1 - X + sqrt((1 - X).^2 + Y.^2);
r2 = -1 - X + sqrt((-1 - X).^2 + Y.^2);
V_true = k .* p .* log(r1 ./ r2);

%% 3. Numerical Approximation & Error Analysis
% Array containing the different segment counts to be tested
segment_counts = [20, 50, 100];
figure_indices = [1, 2, 3];

```

```

for j = 1:length(segment_counts)
    N = segment_counts(j);
    dL = 2 / N; % Length of each
    segment (Total length = 2m)
    dQ = p * dL; % Charge per segment
    V_num = zeros(size(X)); % Initialize
    numerical potential matrix

    % Method of Infinitesimals: Summation of
    discrete point charges
    for i = 1:N
        % Identify the center position of the
        current segment
        x_pos = -1 + (i - 0.5) * dL;
        R = sqrt((X - x_pos).^2 + Y.^2);
        V_num = V_num + (k * dQ ./ R);
    end

    % Calculate the Local Absolute Error (
    Difference)
    V_diff = V_true - V_num;

    %% 4. Visualization of Potential
    Difference (Error)
    figure('Color', 'w');
    mesh(X, Y, V_diff);
    colorbar; % Add color scale for
    quantitative reference
    grid on;

    % Formatting labels and titles
    xlabel('X-axis (m)', 'FontSize', 10);
    ylabel('Y-axis (m)', 'FontSize', 10);
    zlabel('Error \Delta V (V)', 'FontSize',
    10);

    % Dynamic title including segment count
    and student info
    title_str = {'Potential Difference:
    Analytical vs. Numerical (N = ', num2str(N
    ), ')'], ...
    'Author: Yanning He, ID:
    12311512'};
    title(title_str, 'FontSize', 12);

    % Save the result for the report
    file_name = sprintf('figure5.%d.jpg',
    figure_indices(j));
    saveas(gcf, file_name);
end

fprintf('Error analysis plots for N = 20, 50,
    and 100 have been generated.\n');

```

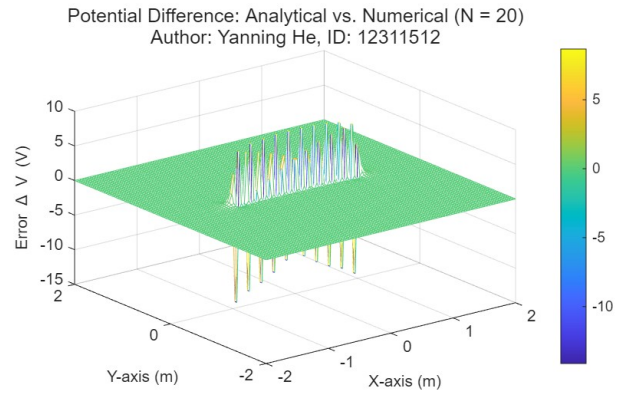


Fig. 13. Potential Difference: Analytical vs. Numerical

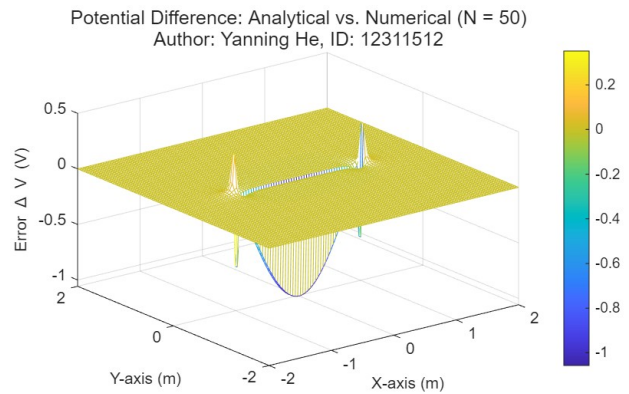


Fig. 14. Potential Difference: Analytical vs. Numerical

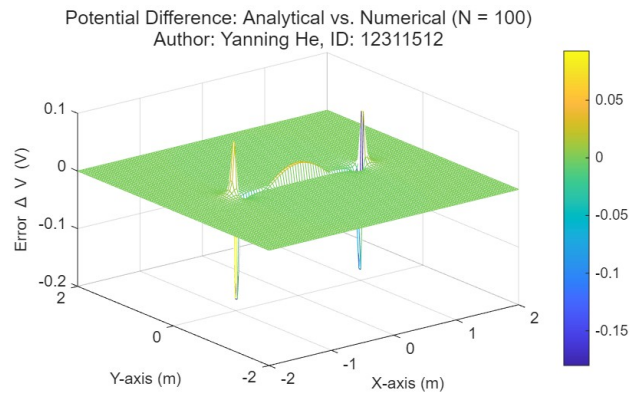


Fig. 15. Potential Difference: Analytical vs. Numerical

First, for 20 segments, 50 segments, and 100 segments, which are discussed above by the graphs, we give a new standard to measure against the precision.

It's easy to show the graph of the difference of electric potential by set the z-axis as $V_{integral} - V_{infinite}$.

Taking a look at the three pictures, we can see that the results of the integral method and the infinitive method produce a large difference at the neighbouring line charge.

On the one hand, this may be due to the fact that the potential gradient at that location is larger and the absolute value is also larger, so the absolute difference value will also be larger.

On the other hand, the infinitive method treats the tiny line charge as a point charge, whereas close to the line charge, the scale difference between the distance from the line charge and the length of the line charge is not large, so it is not possible to make such an approximation.

In addition, at both ends of the line charge, the difference between both methods is the largest of all points.

Comparing these three graphs, we can see that the difference between the two methods decreases significantly as the number of segmentation segments grows.

TABLE II
Error Analysis for Different Segments N

N	dL (m)	dQ (C)	RMS Error (V)
20	0.1	1×10^{-10}	0.76661
50	0.04	4×10^{-11}	0.07510
100	0.02	2×10^{-11}	0.00497

N	MSE (V ²)	MAE (V)	Max Abs Error (V)
20	0.58769	0.076542	14.131
50	0.00564	0.007895	1.0568
100	2.47×10^{-5}	0.000547	0.1801

The analytical solution produces a smooth potential surface and symmetric equipotential contours. The numerical results agree well with the analytical solution in regions away from the line charge, while larger deviations appear near the charged segment because the near-field potential is more sensitive to discretization. The RMS error is 0.7666 for $N=20$, 0.07510 for $N=50$, and 0.004969 for $N=100$. The relative RMS error decreases from 4.0418 to 0.02620, confirming the convergence of the infinitesimal method.

```

clc; clear; close all;

%% 1. Parameter Initialization & Reference
Field
k = 9e9; % Electrostatic
constant (N·m2/C2)
p = 1e-9; % Linear charge
density (C/m)
xm = 2; ym = 2; % Observation domain
boundaries (m)

% Generate calculation grid (100x100 = 10,000
points)
n_points = 100;
x_vec = linspace(-xm, xm, n_points);
y_vec = linspace(-ym, ym, n_points);
[X, Y] = meshgrid(x_vec, y_vec);

% Calculate reference potential using the
Analytical Integral Method
r1 = 1 - X + sqrt((1 - X).^2 + Y.^2);
r2 = -1 - X + sqrt((-1 - X).^2 + Y.^2);

```

```

V_analytical = k .* p .* log(r1 ./ r2);

%% 2. Convergence Study: Error vs. Number of
Segments
segments = 5:5:100; % Array of segment
counts to test
rms_errors = zeros(size(segments)); % Pre-
allocate array for efficiency

for idx = 1:length(segments)
N = segments(idx);
dL = 2 / N; % Length of each
discrete segment
dQ = p * dL; % Charge per segment
V_numerical = zeros(size(X));

% Perform numerical summation for N
segments
for j = 1:N
x_pos = -1 + (j - 0.5) * dL; % Center
of the j-th segment
R = sqrt((X - x_pos).^2 + Y.^2);
V_numerical = V_numerical + (k * dQ ./
R);
end

% Calculate the Root Mean Square (RMS)
Error
% RMS = sqrt( sum( (V_true - V_num)^2 ) /
total_points )
error_sq = (V_analytical - V_numerical)
.^2;
rms_errors(idx) = sqrt(sum(error_sq(:)) /
numel(X));
end

%% 3. Visualization of Convergence Results
figure('Color', 'w');
plot(segments, rms_errors, '-bo', 'LineWidth',
1.5, 'MarkerFaceColor', 'b');
grid on;

% Labeling and Titling for Scientific
Reporting
xlabel('Number of Segments (N)', 'FontSize',
11);
ylabel('Root Mean Square Error (RMS)', '
FontSize', 11);
title_str = {'Convergence Analysis: Potential
Error vs. Discretization Density', ...
'Author: Yanning He, ID: 12311512
'};
title(title_str, 'FontSize', 13);

% Save the convergence plot
saveas(gcf, 'figure5.4.jpg');

fprintf('Convergence analysis complete. RMS
error values have been plotted.\n');

clc; clear; close all;

%% 1. Physical Constants & Grid Initialization
k = 9e9; % Electrostatic
constant (N·m2/C2)
p = 1e-9; % Linear charge
density (C/m)

```

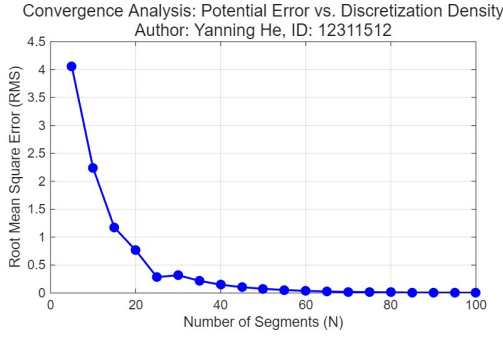


Fig. 16. Convergence Analysis: Potential Error vs. Discretization Density

```

xm = 2; ym = 2; % Observation domain
                boundaries (meters)

% Generate calculation grid (100x100
  resolution)
grid_res = 100;
x_coords = linspace(-xm, xm, grid_res);
y_coords = linspace(-ym, ym, grid_res);
[X, Y] = meshgrid(x_coords, y_coords);

%% 2. Reference Field: Analytical Potential (
  Integral Method)
% Theoretical potential used as the "Ground
  Truth"
r1 = 1 - X + sqrt((1 - X).^2 + Y.^2);
r2 = -1 - X + sqrt((-1 - X).^2 + Y.^2);
V_analytical = k .* p .* log(r1 ./ r2);

%% 3. Convergence Loop: Segment Range 20 to 60
segment_range = 20:1:60;
rms_errors = zeros(size(segment_range)); % Pre
  -allocate for performance

for idx = 1:length(segment_range)
  N = segment_range(idx);
  dL = 2 / N; % Width of each
  discrete element
  dQ = p * dL; % Charge per element
  V_numerical = zeros(size(X));

  % Accumulate potential from each
  infinitesimal segment
  for j = 1:N
    % Precise midpoint calculation for the
    j-th segment in range [-1, 1]
    x_pos = -1 + (j - 0.5) * dL;
    R = sqrt((X - x_pos).^2 + Y.^2);
    V_numerical = V_numerical + (k * dQ ./
    R);
  end

  % Calculate Mean Squared Error (MSE) /
  Deviation
  % Using the user-defined normalization
  factor (1/10000)
  error_sq = (V_analytical - V_numerical)
  .^2;
  rms_errors(idx) = sum(error_sq(:)) / numel
  (X);

```

end

%% 4. Results Visualization & Reporting

```

figure('Color', 'w');
plot(segment_range, rms_errors, '-bo', '
  LineWidth', 1.5, 'MarkerSize', 4);
grid on;

% Academic labeling and student identification
xlabel('Number of Segments (N)', 'FontSize',
  11);
ylabel('Mean Squared Error (MSE)', 'FontSize',
  11);
title_header = 'Convergence Analysis:
  Deviation vs. Discretization Segments';
author_info = 'Author: Yanning He, ID:
  12311512';
title({title_header, author_info}, 'FontSize',
  13);

% Exporting result for technical report
saveas(gcf, 'figure5.5.jpg');

fprintf('High-resolution convergence plot (N
  =20-60) has been generated and saved.\n');

```

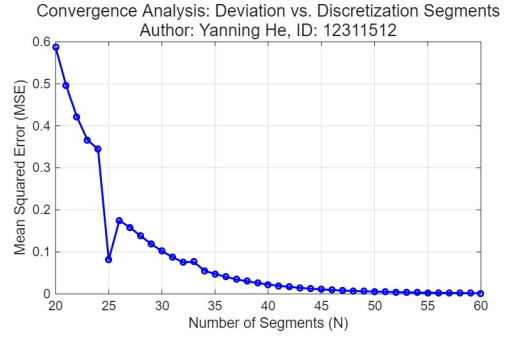


Fig. 17. Convergence Analysis: Deviation vs. Discretization Segments

The RMS can give only one value for each case, which makes it easier to compare the effect of different numbers of segments on the accuracy of the results.

It can be found that, the threshold value may be between 20 and 60.

Through this figure (Fig. 17), we deduce that the threshold may be about 40.

VI. Conclusion

The experiment confirms that the method of infinitesimals is effective for calculating the electrostatic field of a continuous charge distribution. As the number of segments increases, the discrete approximation converges rapidly to the analytical integral solution. The 100-segment result is already very close to the continuous solution, showing that discretization density is a key factor in numerical electromagnetic field analysis.

The infinitive method can generally estimate the potential field of the line charge instead of integral method. It's true that the infinitive method is easier to compute in most

cases. The error is little enough to be ignored when the division is about 40 segments or more, or the testing point is far away from the line charge (the distance between the testing point and the line charge is much bigger than the length of the line charge).

Acknowledgment

data is from these codes.

```
clc; clear; close all;

k = 9e9;
rho = 1e-9;
xm = 2; ym = 2;
n_points = 100;

x = linspace(-xm, xm, n_points);
y = linspace(-ym, ym, n_points);
[X, Y] = meshgrid(x, y);

% Analytical / continuous solution
r1 = 1 - X + sqrt((1 - X).^2 + Y.^2);
r2 = -1 - X + sqrt((-1 - X).^2 + Y.^2);
V_true = k * rho * log(r1 ./ r2);

segment_counts = [20, 50, 100];

N_col = [];
dL_col = [];
dQ_col = [];
RMS_col = [];
MSE_col = [];
MAE_col = [];
MaxAbs_col = [];
RelRMS_col = [];

V_true_rms = sqrt(mean(V_true(:).^2));

for N = segment_counts
    dL = 2 / N;
    dQ = rho * dL;
    V_num = zeros(size(X));

    for i = 1:N
        x_pos = -1 + (i - 0.5) * dL;
        R = sqrt((X - x_pos).^2 + Y.^2);
        V_num = V_num + k * dQ ./ R;
    end

    diff = V_true - V_num;

    N_col(end+1,1) = N;
    dL_col(end+1,1) = dL;
    dQ_col(end+1,1) = dQ;
    RMS_col(end+1,1) = sqrt(mean(diff(:).^2));
    MSE_col(end+1,1) = mean(diff(:).^2);
    MAE_col(end+1,1) = mean(abs(diff(:)));
    MaxAbs_col(end+1,1) = max(abs(diff(:)));
    RelRMS_col(end+1,1) = RMS_col(end) /
        V_true_rms * 100;
end

T = table(N_col, dL_col, dQ_col, RMS_col,
    MSE_col, MAE_col, MaxAbs_col, RelRMS_col,
    ...
```

```
'VariableNames', {'N', 'dL_m', 'dQ_C', '
RMS_error_V', 'MSE_V2', 'MAE_V', '
MaxAbsError_V', 'RelativeRMS_percent'});

disp(T);

writetable(T, 'lab2_error_summary.csv');

points = [-2 -2;
          -2 -1.5;
          -2 -1;
          -2 -0.5];

segment_counts = [100, 50, 20];

Point = ["P1"; "P2"; "P3"; "P4"];
x_m = points(:,1);
y_m = points(:,2);
Distance_to_A_m = sqrt((x_m + 1).^2 + y_m.^2);

V_continuous_V = zeros(4,1);
V_N100_V = zeros(4,1);
V_N50_V = zeros(4,1);
V_N20_V = zeros(4,1);

k = 9e9;
rho = 1e-9;

for idx = 1:4
    X = x_m(idx);
    Y = y_m(idx);

    r1 = 1 - X + sqrt((1 - X)^2 + Y^2);
    r2 = -1 - X + sqrt((-1 - X)^2 + Y^2);
    V_continuous_V(idx) = k * rho * log(r1 /
        r2);

    values = zeros(1,3);
    for s = 1:3
        N = segment_counts(s);
        dL = 2 / N;
        dQ = rho * dL;
        V_num = 0;

        for i = 1:N
            x_pos = -1 + (i - 0.5) * dL;
            R = sqrt((X - x_pos)^2 + Y^2);
            V_num = V_num + k * dQ / R;
        end

        values(s) = V_num;
    end

    V_N100_V(idx) = values(1);
    V_N50_V(idx) = values(2);
    V_N20_V(idx) = values(3);
end

T = table(Point, x_m, y_m, Distance_to_A_m,
    ...
    V_continuous_V, V_N100_V, V_N50_V, V_N20_V
    );

disp(T);

writetable(T, 'lab2_sample_points.csv');
```